



TRABAJO FIN DE MÁSTER

# Desarrollo de una aplicación de gestión de planes de boda

---

**Laura Rosón Lavín**

Tutor: Juan José Ramos Muñoz

Máster en Ingeniería Informática · Universidad de Granada  
Curso 2025/2026

# Tabla de contenidos

|          |                             |         |          |                               |         |
|----------|-----------------------------|---------|----------|-------------------------------|---------|
| <b>1</b> | El problema                 | 1 diap. | <b>4</b> | Implementación                | 3 diap. |
| <b>2</b> | Estado del arte y objetivos | 2 diap. | <b>5</b> | Evaluación y resultados       | 2 diap. |
| <b>3</b> | Diseño de la solución       | 8 diap. | <b>6</b> | Conclusiones y líneas futuras | 2 diap. |

# Organizar una boda es un proyecto largo y disperso

117

invitados de media  
por boda

13

profesionales  
contratados

8-12 meses

de coordinación  
continuada

Y todo esto hay que controlarlo a la vez:

Invitados y acompañantes

Confirmaciones de asistencia

Alergias y menús

Grupos

Mesas y asientos

Tareas con plazos

Eventos de la agenda

Partidas de gasto

Proveedores y contratos

- Todo **depende de todo**: los invitados condicionan las mesas, y las mesas el presupuesto.
- La información acaba **dispersa** en hojas de cálculo, cuadernos y apps separadas.

# Existen plataformas, pero dejan un hueco

|                                              | BODAS.NET   | THE KNOT               | ZOLA            | ESTE PROYECTO            |
|----------------------------------------------|-------------|------------------------|-----------------|--------------------------|
| Organización (invitados, mesas, presupuesto) | Sí          | Sí                     | Sí              | Sí                       |
| Modelo de negocio                            | Marketplace | Marketplace            | Marketplace     | Autocontenido            |
| IA aplicada a...                             | Limitada    | Recomendar proveedores | Redactar textos | Decisiones organizativas |

Su IA trabaja para **su negocio** (vender proveedores), no para las decisiones internas de la pareja. **Aquí está el hueco.**

# Tres objetivos

Desarrollar una aplicación que organice **todos** los aspectos de una boda desde un único panel, con **asistencias de IA** que automaticen las decisiones más repetitivas.

## 1 · Gestión integral

Cubrir en una sola herramienta todo el ciclo: cuenta y bodas, invitados y confirmaciones, grupos y mesas, tareas y agenda, presupuesto y proveedores, y un panel de seguimiento.

OE-1 a OE-6

## 2 · Asistencias con IA

Automatizar las decisiones organizativas más repetitivas mediante propuestas que el usuario revisa y confirma.

OE-7

## 3 · Calidad del producto

Construirla atendiendo a la seguridad, la usabilidad, la robustez frente a errores y la calidad del código.

OE-8

# Tres capas, tres responsabilidades

## Capa de presentación

= El **cliente**, en el navegador  
Lo que el usuario ve y toca · React

↓ pide datos por HTTP ↓

## Capa de aplicación

= El **servidor**, con la lógica  
Decide qué se puede hacer · Node.js + Express

↓ consulta y guarda ↓

## Capa de persistencia

= La **base de datos**  
Donde todo queda guardado · PostgreSQL

### Servicios externos

- Inteligencia artificial
- Envío de correo

Se conectan **solo** con el servidor. El cliente nunca ve sus credenciales.

A continuación, una transparencia por capa: **qué tecnología usa, cómo está diseñada y por qué así.**

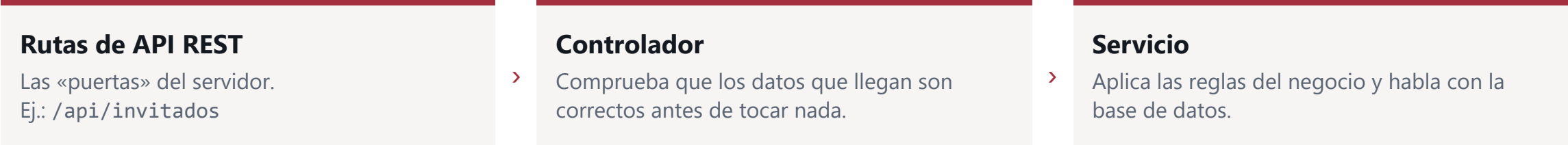
# Capa de presentación: el cliente

| TECNOLOGÍA          | QUÉ ES                                                                 | POR QUÉ LA ELEGÍ                                                                                     |
|---------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>React</b>        | Biblioteca para construir la interfaz con piezas reutilizables.        | Modelo sencillo y el ecosistema más amplio: encontré bibliotecas maduras de calendario y gráficas.   |
| <b>TypeScript</b>   | JavaScript al que se le añaden tipos de datos.                         | Avisa de los errores mientras escribo el código, no cuando ya está en producción.                    |
| <b>Vite</b>         | Herramienta que compila y sirve la aplicación mientras se desarrolla.  | Arranca y refresca los cambios casi al instante, frente a los segundos de las herramientas clásicas. |
| <b>Tailwind CSS</b> | Sistema de estilos mediante clases ya definidas.                       | Da coherencia visual sin escribir hojas de estilo a medida para cada pantalla.                       |
| <b>shadcn/ui</b>    | Colección de componentes accesibles que se copian dentro del proyecto. | Se pueden adaptar libremente, sin depender de una biblioteca externa con estética propia.            |

Decisión clave: **todas las llamadas de red pasan por un único punto**, que añade la sesión, la renueva al caducar y unifica los errores.

# Capa de aplicación: el servidor

Cada petición del cliente recorre siempre el mismo camino:



| TECNOLOGÍA | QUÉ ES                                             | POR QUÉ LA ELEGÍ                                                                                |
|------------|----------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Node.js    | Permite ejecutar JavaScript en el servidor.        | Mismo lenguaje en cliente y servidor: menos cambio de contexto al desarrollar sola.             |
| Express    | Estructura mínima para crear las rutas de la API.  | Maduro y simple; alternativas como NestJS imponen mucha estructura innecesaria aquí.            |
| Zod        | Define qué forma deben tener los datos que entran. | Con una sola definición valida los datos <b>y</b> genera los tipos: no se pueden desincronizar. |

Separar «comprobar» de «hacer» permite **probar la lógica sin levantar un servidor.**

# Capa de persistencia: los datos

## Mi código

«Dame los invitados de esta boda»



## Prisma

Traduce esa orden a lenguaje de base de datos y comprueba que los campos existen.



## PostgreSQL

Guarda la información y garantiza que sea coherente.

## Qué aporta Prisma

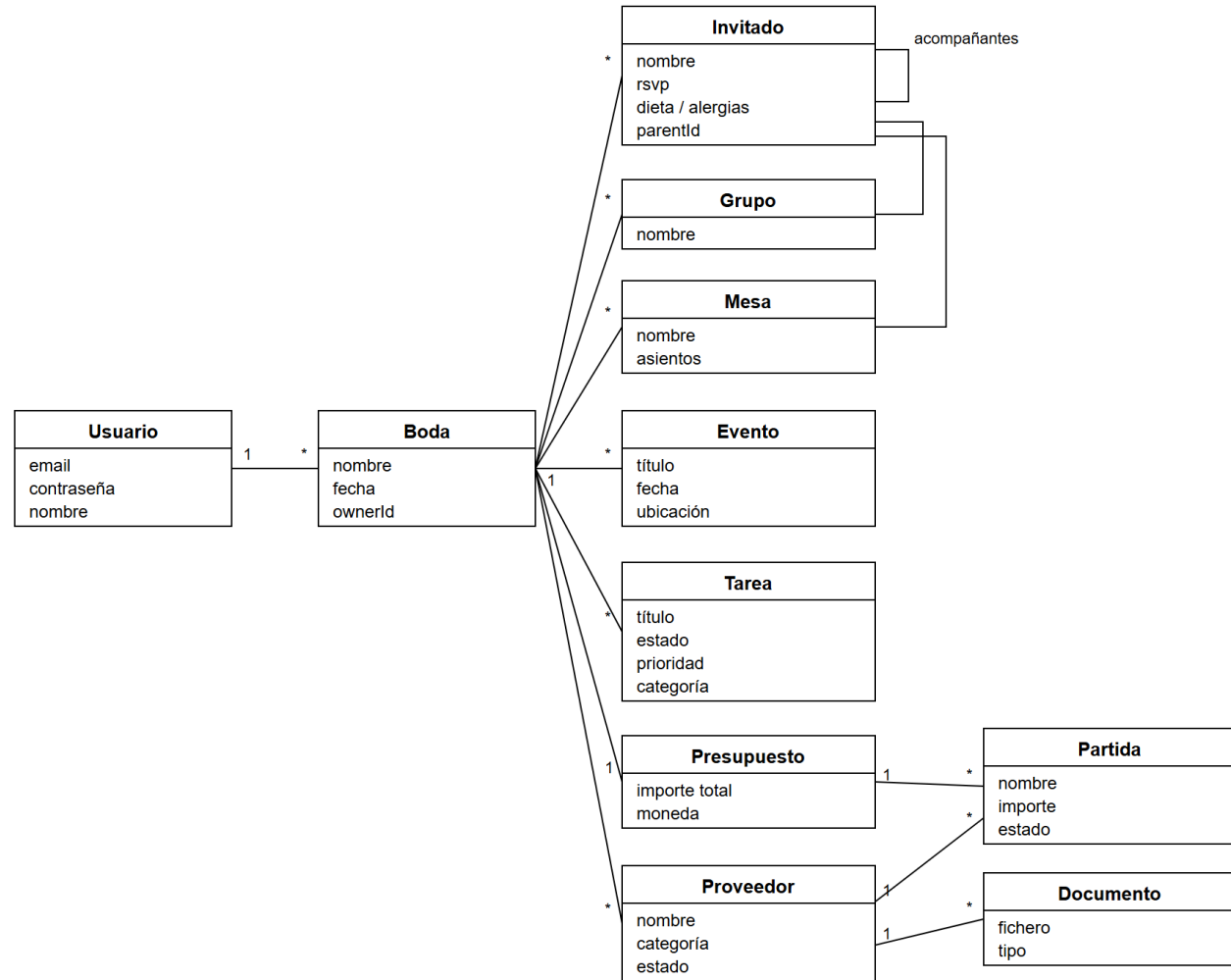
El modelo se declara **una sola vez** en un fichero. A partir de él genera automáticamente el código de acceso y las **migraciones**: cada cambio de la base de datos queda registrado y es repetible.

## Por qué así

Si me equivoco en el nombre de un campo, **falla al compilar**, no delante del usuario. Y las garantías —que un asiento no se repita, que al borrar una boda no queden restos— las impone **PostgreSQL**, no mi código.

# Todo cuelga de la boda

- Cada dato sabe a qué boda pertenece: eso permite el **aislamiento**.
- **Un asiento, una persona**: lo impide la propia base de datos.
- Los **acompañantes** apuntan a su invitado principal.
- Al borrar una boda **no quedan restos**.



# Servicios externos, aislados

```
// Única función que habla con la IA (solo en el servidor)
await openai.chat.completions.create({
  model: "gpt-4o-mini",
  temperature: 0.4,           // baja: respuestas consistentes
  messages: [
    { role: "system", content: reglas },
    { role: "user",   content: contextoDeLaBoda },
  ],
  response_format: { type: "json_schema", ... },
});
```

**Qué se envía:** fecha, número de invitados, nombres y grupos. **Qué no:** correos, teléfonos ni contraseñas.

## Correo

Si hay servidor configurado, envía de verdad; si no, usa una **cuenta de pruebas** que devuelve un enlace de vista previa.

## Por qué aisladas

Cada servicio externo vive en **un único fichero**. Cambiar de proveedor no obliga a tocar el resto, y un fallo suyo **no tumba la aplicación**.

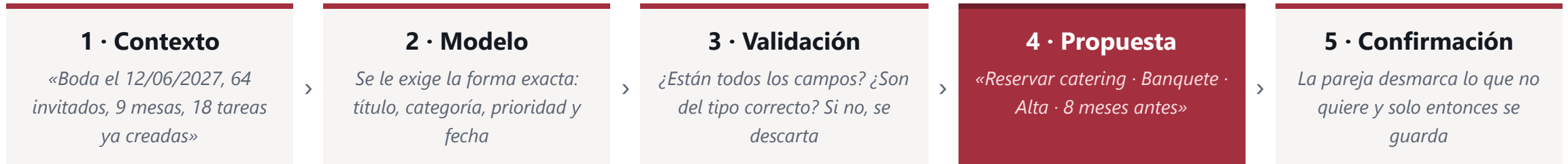
# Seguridad, en todas las capas

Al iniciar sesión se entregan **dos credenciales distintas**:

|                                                                                                                           |                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Token de acceso</b></p> <p>Dura minutos. Viaja en cada petición. Si alguien lo robara, <b>caduca enseguida</b>.</p> | <p><b>Token de refresco</b></p> <p>Dura días y se guarda en la base de datos, así que <b>se puede anular</b>: eso es cerrar sesión de verdad.</p> |
|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|

| MEDIDA                             | QUÉ CONSIGUE                                                                                                          |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Doble comprobación de dueño</b> | Aunque cambies el identificador en la dirección, no puedes ver la boda de otra persona.                               |
| <b>Responder «no encontrado»</b>   | Si respondiera «prohibido», estaría confirmando que esa boda existe. Diciendo «no encontrado» <b>no revelo nada</b> . |
| <b>Contraseñas cifradas</b>        | Se guardan transformadas y no se pueden recuperar, ni siquiera desde la base de datos.                                |
| <b>Arranque seguro</b>             | Si las claves del servidor no son seguras, <b>el servidor no arranca</b> . Mejor eso que arrancar mal.                |

# La IA propone, la pareja decide



## Fiabilidad del formato

Se exige a la IA una respuesta con una **forma cerrada** y se comprueba antes de usarla. Si no encaja, no se utiliza.

## Control del usuario

La IA **nunca escribe en la base de datos**. Devuelve borradores que se guardan solo si la pareja los confirma.

## El contrato con la IA

**Contrato** = acuerdo cerrado sobre la forma exacta que debe tener la respuesta. Se impone y luego se verifica.

```
// 1) Se le exige al modelo que responda con esa forma exacta
response_format: { type: "json_schema", json_schema: { ..., strict: true } }

// 2) Se comprueba la respuesta antes de dársela al usuario
const resultado = esquema.safeParse(respuesta);
if (!resultado.success) throw { status: 502, message: "Formato no esperado" };
```

- Se verifica **aunque el proveedor ya lo garantice**: no delego mis datos en un servicio externo.
- Si la IA falla o no está configurada, **se avisa con un mensaje claro y el resto de la aplicación sigue funcionando** con normalidad.

# El caso difícil: distribuir las mesas

Los identificadores de mesas e invitados se generan en cada boda, así que **no se pueden fijar por adelantado**: el modelo de lenguaje podría inventarse referencias que no existen.

## Propuesta del modelo

Asigna cada invitado a una mesa y un asiento



## Se depura

Se descartan mesas que no existen, asientos fuera de capacidad y repetidos



## Propuesta limpia

Solo entonces se le enseña a la pareja

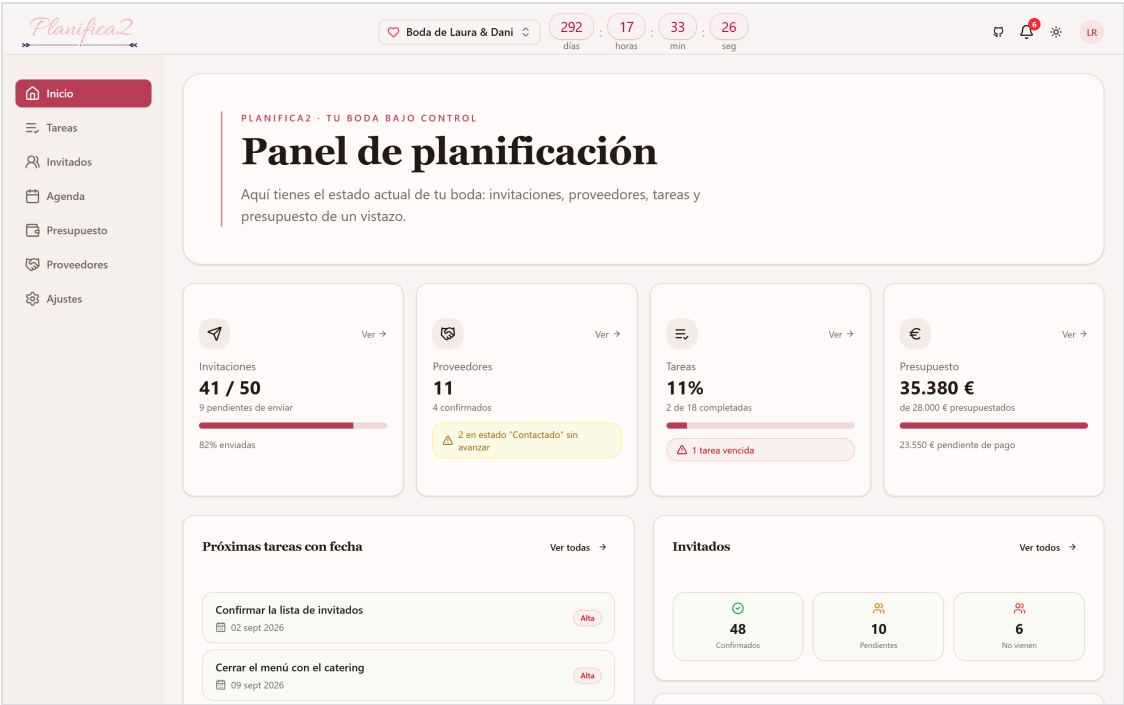


## Se aplica entera

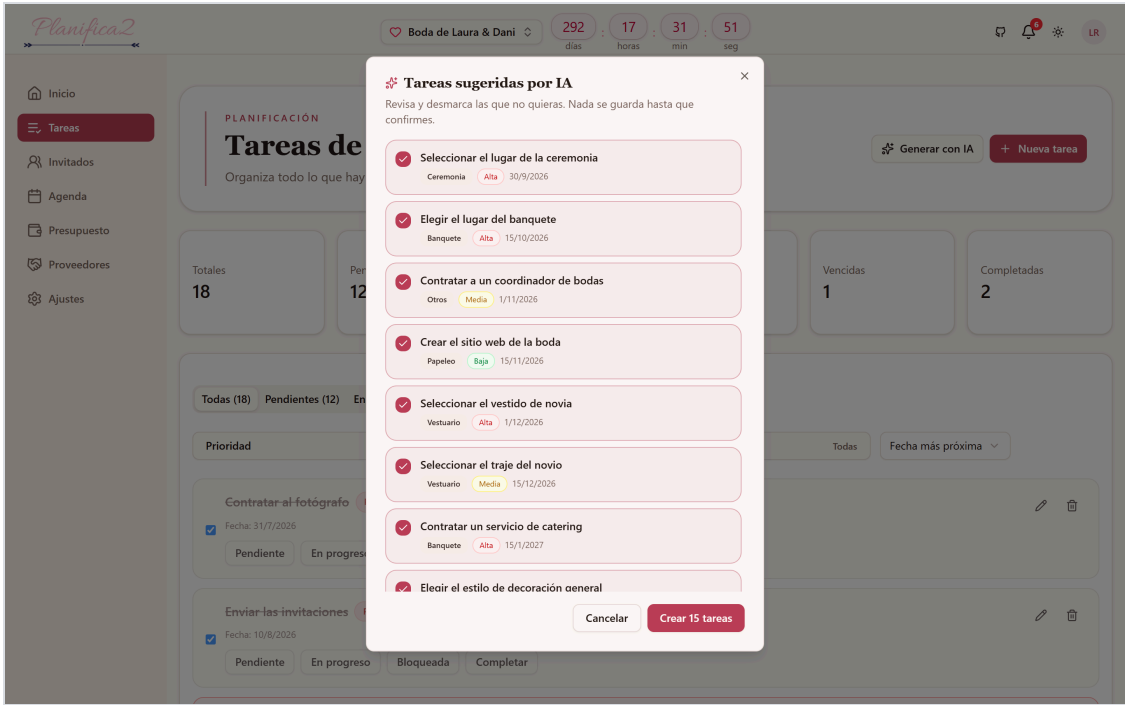
En un solo bloque: o se coloca todo, o no se toca nada

Nunca se confía en la salida del modelo de lenguaje: **se contrasta con lo que hay realmente en la base de datos.**

# La aplicación en funcionamiento



Panel de seguimiento: estado global de la boda y cuenta atrás



Propuesta de tareas generada por la IA, editable antes de confirmar

# Cómo se ha verificado

**Entorno:** servidor Node.js contra una base de datos PostgreSQL en contenedor · pruebas con Vitest y supertest · 64 invitados de prueba.

| TIPO DE PRUEBA       | QUÉ COMPRUEBA                                                                                  | RESULTADO   |
|----------------------|------------------------------------------------------------------------------------------------|-------------|
| Unitarias            | Cifrado de contraseñas, credenciales de sesión, plantilla del correo y tratamiento de errores. | Superadas   |
| Integración          | Recorrido completo registro → boda → invitado, con peticiones reales contra la base de datos.  | Superadas   |
| Aislamiento          | Acceder a una boda ajena devuelve «no encontrado», sin revelar que existe.                     | Superada    |
| Estilo y compilación | Análisis de estilo y compilación de cliente y servidor.                                        | Sin errores |
| Integración continua | Todo lo anterior se ejecuta solo con cada cambio del proyecto.                                 | En verde    |

**18 pruebas automatizadas** en 5 ficheros, todas superadas.

# Lo que hace la aplicación

## Invitados

Alta con acompañantes, confirmación de asistencia, alergias y menús, invitaciones por correo, página pública de confirmación, importar y exportar en CSV.

## Economía

Presupuesto con partidas, importes estimado, real y pagado, gráficas de reparto, y proveedores con documentos adjuntos.

## Organización

Grupos de invitados, mapa de mesas con asignación de asientos, tareas con prioridad y plazo, y agenda con calendario y cuenta atrás.

## Seguimiento

Panel con el estado global de la boda y avisos de tareas y pagos vencidos o próximos.

## Asistencias con inteligencia artificial

Genera el **listado de tareas** a partir de la fecha y el número de invitados · Propone la **distribución de las mesas** respetando grupos y capacidad · Reparte el **presupuesto por categorías** y sugiere proveedores. Siempre como propuesta editable.

# Conclusiones

- Se ha construido una aplicación **funcional y autocontenida** que cubre el ciclo completo de organización de una boda, sin capa comercial de por medio.
- La aportación diferencial no es usar IA, sino **cómo**: un patrón de **propuesta verificada y bajo control del usuario**, aplicado a las decisiones organizativas.
- Los **ocho objetivos** planteados se han alcanzado, respaldados por 18 pruebas automatizadas e integración continua.
- **Limitación reconocida**: el cliente todavía no cuenta con pruebas automatizadas propias.

# Líneas de trabajo futuro

## 1 · Pruebas del cliente

Cubrir la interfaz con pruebas automatizadas, la limitación principal del trabajo.

## 3 · Aplicación móvil

Consulta y avisos desde el teléfono en cualquier momento.

## 5 · Gestión compartida

Que los dos miembros de la pareja gestionen la misma boda a la vez.

## 2 · IA conversacional

Permitir refinar las propuestas hablando con el asistente, en lugar de aceptarlas o descartarlas.

## 4 · Exportación a PDF

Distribución de mesas, agenda y resumen del presupuesto en documento imprimible.

## 6 · Despliegue y cumplimiento

Puesta en producción con copias de seguridad y cumplimiento formal de protección de datos.

# Gracias

---

¿Alguna pregunta?

Laura Rosón Lavín · Máster en Ingeniería Informática · Universidad de Granada